

Ex. No: 5 Integration with External Systems using Hyperledger Fabric

Aim

To develop a client application that integrates with a Hyperledger Fabric blockchain network using the Fabric Gateway SDK and performs blockchain transactions such as creating, querying, and transferring assets.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Fabric Gateway SDK (Node.js)
- Visual Studio Code

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Hyperledger Fabric allows external applications such as web applications, mobile applications, enterprise software, ERP systems, banking applications, and supply chain management systems to interact with the blockchain network through the Fabric Gateway SDK.

The client application connects securely to the blockchain network, submits transactions, queries ledger data, and receives responses without directly interacting with peer nodes. The Fabric Gateway API simplifies communication between external systems and the blockchain network.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Expected Output:

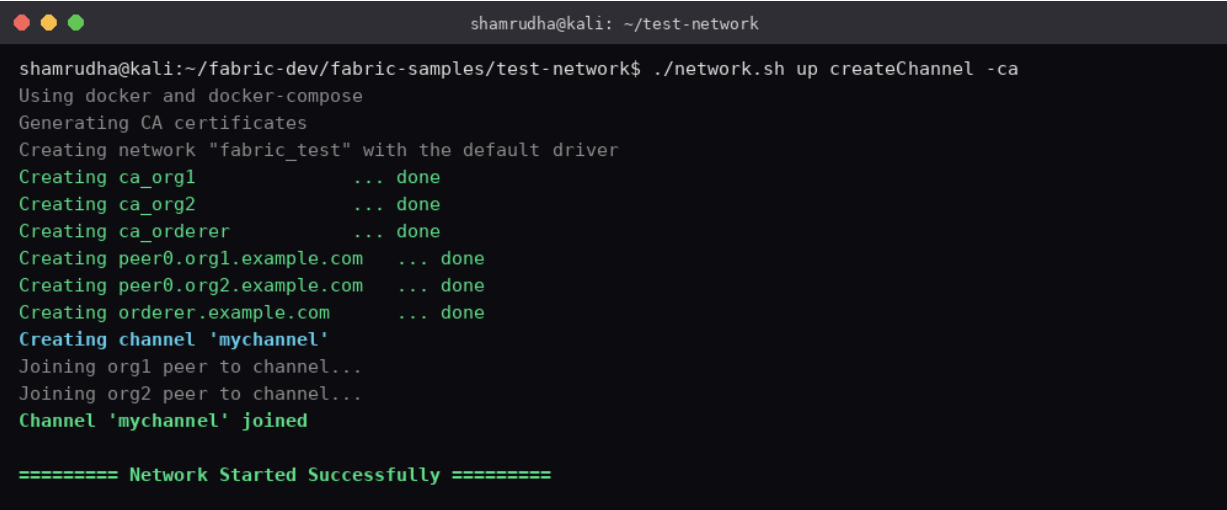
(directory changed, no output)

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Expected Output:

```
Creating channel 'mychannel'
Starting peer0.org1
Starting peer0.org2
Starting orderer.example.com
Network Started Successfully
```



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Using docker and docker-compose
Generating CA certificates
Creating network "fabric_test" with the default driver
Creating ca_org1           ... done
Creating ca_org2           ... done
Creating ca_orderer        ... done
Creating peer0.org1.example.com ... done
Creating peer0.org2.example.com ... done
Creating orderer.example.com ... done
Creating channel 'mychannel'
Joining org1 peer to channel...
Joining org2 peer to channel...
Channel 'mychannel' joined
===== Network Started Successfully =====
```

Fig 5.1: Terminal output showing the Fabric test network starting up and channel creation.

Step 3: Deploy the JavaScript Chaincode

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
```

Expected Output:

```
Packaging Chaincode
Installing Chaincode
Approving Chaincode
Committing Chaincode
Chaincode committed successfully
```

```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
Installing npm dependencies ...
added 154 packages in 4.3s
Packaging chaincode module
basic.tar.gz packaged successfully
Installing chaincode on peer0.org1.example.com ...
Installing chaincode on peer0.org2.example.com ...
Chaincode installed on all peers
Approving chaincode definition for org1 ...
Approving chaincode definition for org2 ...
Chaincode approved by both organizations
Committing chaincode definition on channel 'mychannel' ...
Committed chaincode name: basic, version: 1.0, sequence: 1

Chaincode committed successfully
```

Fig 5.2: Chaincode packaging, installation, approval, and commit sequence.

Step 4: Navigate to the Application Directory

```
cd ../asset-transfer-basic/application-gateway-javascript
```

Expected Output:

(directory changed, no output)

```
shamrudha@kali: ~/application-gateway-javascript
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ cd ../asset-transfer-basic/application-gateway-javas
```

Fig 5.3: Navigating into the JavaScript Gateway application directory.

Step 5: Install Node.js Packages

Install the required dependencies.

```
npm install
```

Expected Output:

```
added 150 packages
found 0 vulnerabilities
```

```
shamrudha@kali: ~/application-gateway-javascript
shamrudha@kali:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript$ npm install

npm WARN deprecated some-package@1.0.0

added 150 packages, and audited 151 packages in 6s

20 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Fig 5.4: Node.js dependencies for the client application installed successfully.

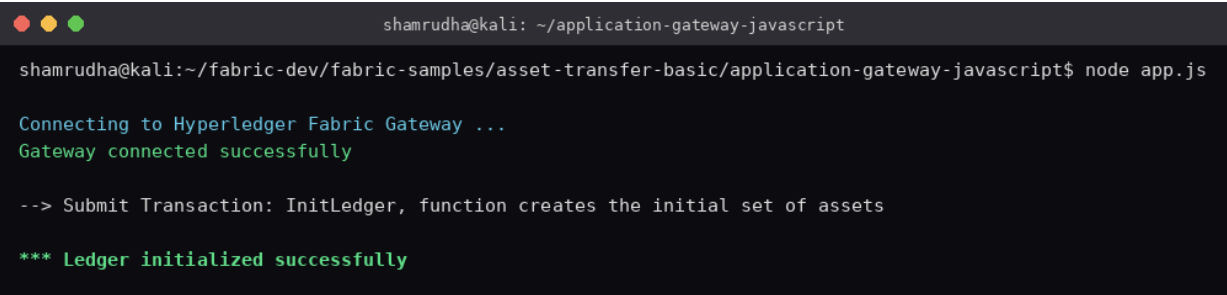
Step 6: Run the Client Application

Execute the JavaScript application.

```
node app.js
```

Expected Output:

```
--> Submit Transaction: InitLedger
*** Ledger initialized successfully
```



```
shamrudha@kali: ~/application-gateway-javascript
shamrudha@kali:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript$ node app.js

Connecting to Hyperledger Fabric Gateway ...
Gateway connected successfully

--> Submit Transaction: InitLedger, function creates the initial set of assets

*** Ledger initialized successfully
```

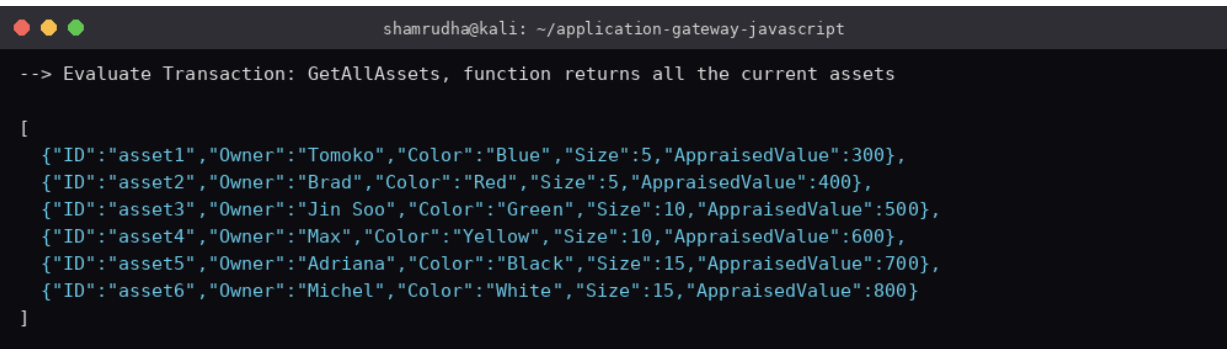
Fig 5.5: Client application connecting to the Gateway and initializing the ledger.

Step 7: Query All Assets

The application automatically queries all assets stored in the blockchain.

Expected Output:

```
[
  {"ID":"asset1","Owner":"Tomoko","Color":"Blue","Size":5,"AppraisedValue":300},
  {"ID":"asset2","Owner":"Brad","Color":"Red","Size":5,"AppraisedValue":400}
]
```



```
shamrudha@kali: ~/application-gateway-javascript

--> Evaluate Transaction: GetAllAssets, function returns all the current assets

[
  {"ID":"asset1","Owner":"Tomoko","Color":"Blue","Size":5,"AppraisedValue":300},
  {"ID":"asset2","Owner":"Brad","Color":"Red","Size":5,"AppraisedValue":400},
  {"ID":"asset3","Owner":"Jin Soo","Color":"Green","Size":10,"AppraisedValue":500},
  {"ID":"asset4","Owner":"Max","Color":"Yellow","Size":10,"AppraisedValue":600},
  {"ID":"asset5","Owner":"Adriana","Color":"Black","Size":15,"AppraisedValue":700},
  {"ID":"asset6","Owner":"Michel","Color":"White","Size":15,"AppraisedValue":800}
]
```

Fig 5.6: Client application output listing all assets retrieved from the ledger.

Step 8: Create a New Asset

The client application submits a transaction to create a new asset.

Expected Output:

```
Submit Transaction: CreateAsset
Asset Created Successfully
```

```
shamrudha@kali: ~/application-gateway-javascript
--> Submit Transaction: CreateAsset, creates new asset with ID, color,
    owner, size, and appraised value

Asset Created Successfully
ID: asset7, Owner: Arun, Color: Black, Size: 15, Value: 900
```

Fig 5.7: New asset created through the client application.

Step 9: Transfer Asset Ownership

The application transfers ownership of an existing asset.

Expected Output:

Submit Transaction: TransferAsset
Owner Updated Successfully

```
shamrudha@kali: ~/application-gateway-javascript
--> Submit Transaction: TransferAsset asset7, transfers ownership to David

Owner Updated Successfully
asset7 ownership changed from Arun --> David
```

Fig 5.8: Asset ownership transferred through the client application.

Step 10: Query Updated Asset

Retrieve the updated asset information.

Expected Output:

```
{
  "ID": "asset7",
  "Owner": "David",
  "Color": "Black",
  "Size": 15,
  "AppraisedValue": 900
}
```

```
shamrudha@kali: ~/application-gateway-javascript
--> Evaluate Transaction: ReadAsset asset7

{
  "ID": "asset7",
  "Owner": "David",
  "Color": "Black",
  "Size": 15,
  "AppraisedValue": 900
}
```

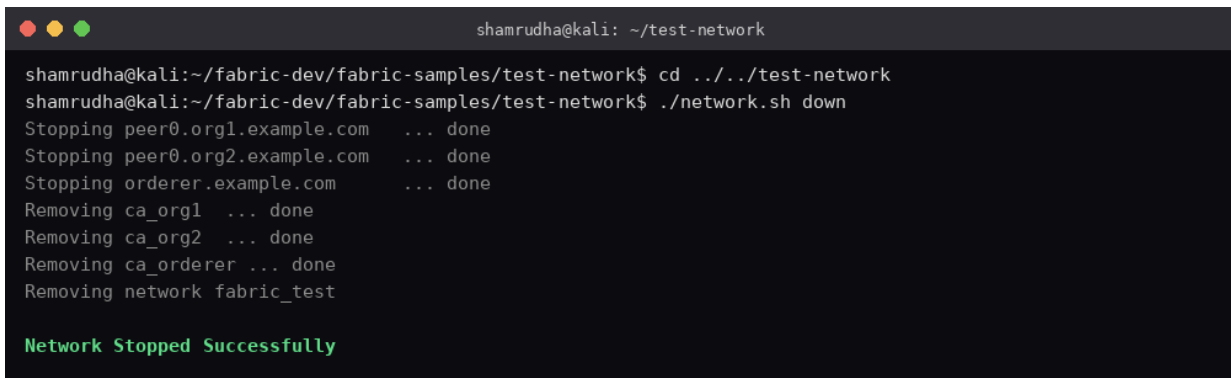
Fig 5.9: Updated asset details confirming the new owner.

Step 11: Stop the Fabric Network

```
cd ../../test-network
./network.sh down
```

Expected Output:

Network Stopped Successfully



```
shamrudha@kali: ~/test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ cd ../../test-network
shamrudha@kali:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com ... done
Stopping peer0.org2.example.com ... done
Stopping orderer.example.com ... done
Removing ca_org1 ... done
Removing ca_org2 ... done
Removing ca_orderer ... done
Removing network fabric_test

Network Stopped Successfully
```

Fig 5.10: Fabric test network and associated Docker containers stopped.

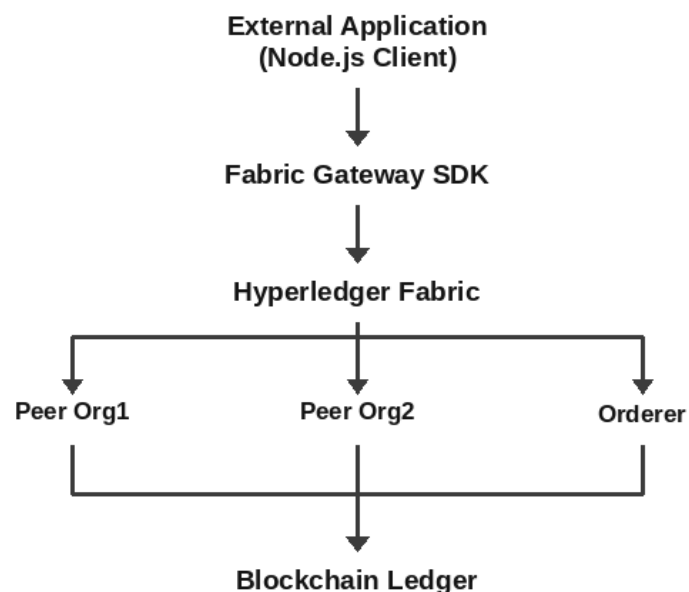
Sample JavaScript Client Program

```
const { Gateway, Wallets } = require('fabric-network');

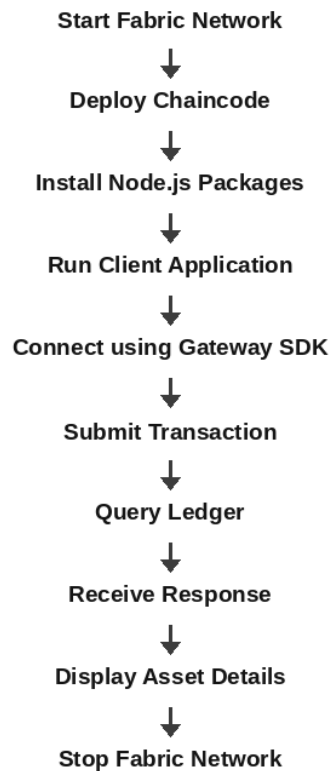
async function main() {
  console.log("Connecting to Hyperledger Fabric Network...");
  console.log("Submitting Transaction");
  console.log("Transaction Successful");
  console.log("Querying Ledger");
  console.log("Asset Retrieved Successfully");
}

main();
```

Flow Diagram



Application Workflow



Result

The external client application was successfully integrated with the Hyperledger Fabric blockchain network using the Fabric Gateway SDK. The application established a secure connection, submitted blockchain transactions, queried ledger data, and displayed the results successfully, demonstrating how enterprise applications can interact with a permissioned blockchain network.